

CodeSage-AI
Master Test Plan

Version 1.0

An AI-Powered Technical Debt Analytics Dashboard for Agile Small-Scale Software Teams

Group 16
Project ID 7

CS3203 - Software Engineering Project
Mentor: Mr. Anju Chamantha

Date: 27th September 2026

Revision History

Date	Version	Description	Author
27/Sep/26	1.0	Master test plan for CodeSage-AI, aligned with SRS v1.1 [1] and SAD v1.1 [2]	Group 16

Individual Contributions

Index No.	Name	Contributions
230432G	Nethmini R.M.N.	Prepared and documented Evaluation Mission and Test Motivation, Target Test Items, Configuration Testing, Machine Learning Model Testing.
230435T	Nethsara U.D.K.C.	Prepared and documented Data and Database Integrity Testing, Function Testing, Security and Access Control Testing, Security and Access Control Testing, Failover and Recovery Testing, Risks, Dependencies, Assumptions, and Constraints.
230451M	Pabasara H.H.J.	Prepared and documented User Interface Testing, Performance Profiling, Performance Profiling, Test Evaluation Summaries, Reporting on Test Coverage.

Table of Contents

Table of Contents	3
1. Evaluation Mission and Test Motivation	4
2. Target Test Items	6
3. Test Approach	7
3.1 Testing Techniques and Types	9
3.1.1 Data and Database Integrity Testing	9
3.1.2 Function Testing	13
3.1.3 User Interface Testing	15
3.1.4 Performance Profiling	17
3.1.5 Load Testing	19
3.1.6 Security and Access Control Testing	20
3.1.7 Failover and Recovery Testing	24
3.1.8 Configuration Testing	28
3.1.9 Machine Learning Model Testing	32
4. Deliverables	36
4.1 Test Evaluation Summaries	36
4.2 Reporting on Test Coverage	36
5. Risks, Dependencies, Assumptions, and Constraints	37
5.1 Risks	37
5.2 Dependencies	38
5.3 Assumptions	39
5.4 Constraints	39
6. References	40

1. Evaluation Mission and Test Motivation

CodeSage AI is an AI-assisted technical debt analytics platform that connects to software repositories, analyzes source code, and helps developers identify maintainability risks. The system focuses mainly on detecting self-admitted technical debt from source-code comments and estimating defect risk using machine-learning models. These results are presented through an interactive dashboard so that developers can inspect findings, prioritize remediation work, and monitor the health of a codebase over time.

Because CodeSage AI combines a web application, background processing, database storage, external services, and machine-learning components, defects in one part of the system can affect the reliability of the entire analysis process. Incorrect findings, incomplete scan results, cross-workspace data exposure, authentication failures, or inconsistent scoring could reduce trust in the platform. Therefore, a structured testing process is required to verify both the functional behavior of the application and the quality attributes expected from the system.

The system is built using a Next.js frontend, FastAPI backend services, PostgreSQL, Redis, background workers, and a separate ML inference service, with authentication provided through Asgardeo. The test process must therefore cover each major layer of the architecture, including the user interface, API services, scan pipeline, database, security controls, deployment environment, and ML-related processing.

The main objectives of the testing process are

- **Identify defects before deployment:** Testing is carried out across frontend, backend, database, background-processing, integration, and deployment layers so that functional and integration defects can be detected as early as possible.
- **Verify functional requirements:** The main user workflows, including authentication, repository connection, scanning, cancellation, technical debt detection, scoring, dashboard viewing, scan history, profile management, and workspace administration, are tested to confirm that they behave according to the Software Requirement Specification.
- **Protect data integrity and tenant isolation:** Since CodeSage AI supports multiple workspaces, testing verifies that data remains isolated between tenants, finalized analysis results remain consistent and immutable, failed operations do not leave partial data, and deletion operations do not affect unrelated records.
- **Assess security and access control:** Authentication, session handling, role-based authorization, secret protection, audit logging, input validation, and database-level access controls are tested to reduce the risk of unauthorized access or sensitive-data exposure.
- **Verify ML-related behavior:** The SATD and bug-risk pipelines are tested to ensure that expected inputs and outputs are handled correctly, prediction values and model provenance are stored consistently, invalid model responses are rejected safely, and unavailable ML services result in controlled degraded behavior.
- **Evaluate reliability and recovery:** The system is tested under scan failures, worker interruptions, unavailable external services, component restarts, and database recovery scenarios to verify that durable data is not corrupted and that the application can return to an operational state.
- **Assess performance and workload behavior:** API responsiveness, scan processing, background workers, database activity, and concurrent workloads are evaluated against the defined test environment and workload baseline.

- **Verify user-interface quality and usability:** The main interfaces are tested for correct data presentation, error and empty states, keyboard operation, accessibility, supported viewport behavior, and completion of the primary user journey.
- **Validate configuration and deployment behavior:** The application is tested across local, CI, and production-like environments to confirm that supported runtime settings, database migrations, service configurations, and deployment components behave as expected.
- **Provide confidence to stakeholders:** The overall purpose of testing is to provide evidence that CodeSage AI produces consistent, traceable, and useful analysis results and that the platform satisfies its functional and non-functional requirements before release.

2. Target Test Items

The table below identifies all software components, services, and supporting elements that have been identified as targets for testing. Items are grouped by category.

Item	What We Test
Frontend & User Interface	Main screens navigation dashboard finding details scan history scoring profiles workspace and team views themes responsive layout keyboard navigation and accessibility
Authentication & Sessions	Asgardeo OIDC authentication callback handling session creation and validation expiry logout cookie security and authentication-state handling
Workspaces Members & RBAC	Workspace management and isolation invitations memberships role changes role-based permissions and role-aware operations
Repository & GitHub Integration	Repository connection and validation metadata and branch retrieval cloning duplicate handling GitHub failures and rate-limit behavior
Backend API	FastAPI endpoints request validation response contracts authentication dependencies error handling and request-to-storage behavior
Scan Pipeline	Scan initiation repository cloning metric extraction debt detection scoring finalization cancellation progress tracking failure handling and scan lifecycle transitions
Technical Debt Detection	Rule-based findings PMD findings security patterns SATD detection categories severity evidence and finding provenance
ML Models & Inference	ML feature contracts SATD classification class-level bug-risk prediction file-level risk aggregation model versions model artifact validation degraded operation and prediction persistence
Scoring & Prioritization	Finding priorities category weights bug-risk contribution health scores grades security-priority behavior scoring profiles and rescoreing
Dashboard & Analysis History	Health summaries category breakdowns trends Refactor-First results file tree finding details historical snapshots and reconstructed scores
Database & Data Integrity	PostgreSQL schema constraints transactions Row-Level Security tenant isolation immutable results deletion behavior audit records migrations and historical-data integrity
Background Processing & Redis	Celery task execution queue behavior worker interruption and redelivery scan progress cancellation state Redis transient data and score-cache processing
Security & Audit	Authorization cross-workspace access prevention database privileges sensitive-data protection session revocation audit logging input validation and security headers
External Service Integrations	GitHub Asgardeo Resend and ML-service success failure malformed-response and unavailable-service behavior

Deployment & Configuration	Docker Compose k3s deployment Neon PostgreSQL environment variables service configuration runtime and browser configurations networking and KEDA autoscaling [18]
Failover & Recovery	Application and worker failures degraded services transaction rollback PostgreSQL backup and restore Neon point-in-time recovery post-recovery integrity and recovery time
Performance & Load	API response times database performance concurrent scans worker capacity resource usage and system behavior under the defined workload baseline

3. Test Approach

The test approach describes the overall strategy used to verify the quality, reliability, security, and correct behavior of CodeSage AI. Testing is carried out at different levels, including unit, integration, database, user-interface, performance, security, recovery, configuration, and ML-model testing. Automated testing is used where repeatability is important, while selected deployment, recovery, browser, and usability checks are performed manually.

Testing is carried out at several levels to verify CodeSage AI from individual components up to complete user workflows. Unit testing is used to verify backend services, extractors, detectors, scoring logic, and frontend components in isolation. Integration testing verifies interactions between API endpoints, PostgreSQL, the scan pipeline, authorization, persistence, and ML-service clients. Frontend testing verifies component behavior, user interactions, and UI states using Vitest and React Testing Library. End-to-end testing uses Playwright to verify complete user journeys through the web application. Selected deployment, browser compatibility, recovery, and usability checks are performed manually in the production-like environment.

Data and Database Integrity Testing

Data and database integrity testing verifies that analysis results, user data, workspace data, scoring profiles, sessions, and audit information are stored and managed correctly. PostgreSQL 16 [4] is used for integration testing, with isolated databases created through Docker and Testcontainers [5].

The actual Alembic [6] migration chain is applied when creating the test databases so that schema constraints, roles, grants, and Row-Level Security policies can be tested under conditions close to the deployed environment. Tests also verify transaction rollback, result immutability, tenant isolation, deletion behavior, and the absence of orphaned records.

Function Testing

Function testing verifies that the main features of CodeSage AI behave according to the Software Requirement Specification. This includes authentication, repository connection, scan execution and cancellation, technical debt detection, scoring, dashboard behavior, scan history, scoring profiles, and role-based operations.

Backend unit and integration tests are implemented using pytest and FastAPI's TestClient [7]. Vitest and React Testing Library are used for frontend component testing, while Playwright is used to test complete browser-based user journeys. Boundary-value analysis, state-transition testing, decision tables, and expected-output comparison are also used where appropriate.

User Interface Testing

User interface testing verifies that the main screens correctly display API data and provide suitable loading, empty, error, and success states. The Projects, Dashboard, Finding Detail, Scan History, Profiles, Workspace, and Team interfaces are tested.

Vitest and React Testing Library are used for component-level testing, while Playwright is used for end-to-end browser testing. Accessibility checks are performed using axe-core and keyboard-navigation tests. Selected checks, including real authentication, themes, browser compatibility, and usability, are also carried out manually on the deployed application.

Performance Profiling

Performance profiling is used to identify parts of the system that may cause excessive response time or resource usage. Particular attention is given to API response times, database queries, scan processing, metric extraction, technical-debt detection, scoring, and dashboard retrieval.

Performance measurements are carried out under a documented test environment so that results can be compared consistently. Database behavior and application-resource usage are monitored to identify bottlenecks that may require query optimization, indexing, or changes to processing logic.

Load Testing

Load testing evaluates how CodeSage AI behaves when multiple users, workspaces, or scans are active at the same time. The tests focus on API responsiveness, scan-processing capacity, database behavior, and worker scaling under increased workload.

Load tests are performed using defined workload baselines rather than attempting to prove unlimited capacity. Worker autoscaling in the deployed k3s environment is also observed to verify that KEDA responds to queued scan workloads as expected.

Security and Access Control Testing

Security and access control testing verifies authentication, role-based authorization, workspace isolation, session handling, input validation, secret protection, audit logging, and database privileges.

Automated negative tests are used to confirm that unauthorized requests, invalid sessions, cross-workspace access, and restricted database operations are rejected correctly. Role permissions are compared with policy.json, while PostgreSQL Row-Level Security is tested using the restricted application role. Deployed response headers and other environment-specific security behavior are checked separately.

Failover and Recovery Testing

Failover and recovery testing verifies that component failures do not corrupt durable data and that CodeSage AI can recover to an operational state. Automated fault-injection tests simulate failures during scan execution, ML-service communication, cancellation, timeout handling, and database finalization.

Manual recovery drills are performed in the production-like k3s environment by restarting or stopping selected components. Database recovery is tested using Neon point-in-time recovery and PostgreSQL backup and restore procedures. Recovery time is measured against the 30-minute target defined by REL-07.

Configuration Testing

Configuration testing verifies that CodeSage AI behaves correctly under supported application, database, scoring, browser, runtime, and deployment configurations. Tests cover configuration such as authentication settings, cookie behavior, scoring parameters, ML-service availability, database migration paths, and browser/display settings.

The application is tested in local Docker Compose, CI, and production-like k3s environments. Automated tests are primarily executed in Chromium, while Firefox, Edge, and Safari are checked manually where required.

ML Model Testing

ML model testing verifies the behavior and integration of the bug-proneness and SATD models used by CodeSage AI. Tests validate the canonical ML input features, model artifact compatibility, prediction ranges, class-to-file risk aggregation, supported SATD categories, prediction persistence, model provenance, and degraded behavior when ML services are unavailable.

The bug-risk model is also evaluated using project-separated validation to reduce cross-project leakage. Automated tests verify ML request and response contracts, feature extraction, persistence, and failure handling, while recorded offline evaluation metrics are used to describe model performance. These evaluation metrics are treated as experimental evidence rather than fixed acceptance thresholds unless a specific requirement defines one.

3.1 Testing Techniques and Types

3.1.1 Data and Database Integrity Testing

Overview:

Data and database integrity testing verifies that CodeSage AI stores, protects, updates, and removes data correctly while maintaining consistency, traceability, and isolation at the database level.

Technique Objective:	Verify that CodeSage AI maintains database integrity throughout the full data lifecycle. The testing focuses on ensuring that analysis results are stored completely and consistently, finalized results remain immutable, scan finalization is atomic, workspace data remains isolated, deletion operations do not leave unintended or orphaned records, credentials are not stored in a reusable form, security events are preserved in an append-only audit trail, and historical results can be reconstructed from stored data.
----------------------	---

Technique:	• Least-privilege execution: ○ Application operations are executed using the <code>codesage_app</code> role with the relevant workspace context, matching the production access model. • Real pipeline with controlled dependencies ○ The actual scan and finalization logic is executed while Git, extractors, ML services, and Redis are replaced with controlled stubs where necessary. • API-level testing ○ Endpoints are called using FastAPI's TestClient to verify their effect on stored data • Negative privilege testing ○ Restricted SQL operations are executed using the application role and must be rejected by PostgreSQL. • Database catalog checks ○ PostgreSQL metadata is inspected to verify privileges, column definitions, constraints, and other schema properties. • Orphan detection ○ Foreign-key relationships are checked after deletion operations to confirm that no rows reference missing parent records. • Row and checksum comparison ○ Data is recorded before and after selected operations to confirm that only the intended rows are changed.
------------	--

Oracles:	• Alembic migrations complete successfully and produce the expected schema. • Database constraints reject invalid, duplicate, or orphaned records. • Database rows and relationships match the expected state. • No orphaned records remain after deletion. • RLS correctly isolates workspace data. • Role and permission tables match <code>policy.json</code>. • Invalid operations return the expected HTTP or PostgreSQL error codes. • Finalized scan data remains immutable. • Failed transactions and partial finalization attempts roll back completely. • Scan attempts record the correct status, timestamps, trigger, and failure details. • Repeated scans produce consistent fingerprints and model-version references. • Finding provenance, category, severity, description, and confidence are stored correctly. • Model-version records remain preserved when repositories are deleted. • Profile changes and scoring history produce the expected stored and derived results. • Security-related operations create the expected append-only audit records. • Session and invitation credentials are stored only in hashed or non-reusable form. • Sign-out or membership revocation invalidates the corresponding session. • Row-set and checksum comparisons confirm that only the intended data changed.
----------	--

Required Tools:	• PostgreSQL 16 (postgres:16-alpine) - Database under test, matching the major version used in production. • pytest – Runs the automated unit and integration tests. • Testcontainers (Python) – Creates isolated PostgreSQL containers for integration testing. • Alembic – Builds test databases using the actual migration chain. • SQLAlchemy 2 + psycopg 3 – Provides database access for the application and test code. • FastAPI TestClient [7] (httpx) – Calls API endpoints during integration testing. • Docker – Provides the container runtime for test databases. • GitHub Actions – Runs migration, tenant-isolation, and other automated test suites in CI.
Success Criteria:	• All database migrations complete successfully and produce the expected schema. • Workspace data remains correctly isolated through RLS. • Database constraints prevent invalid, duplicate, and orphaned records. • Finalized analysis results remain immutable. • Failed transactions and scan finalization operations leave no partial data. • Deletion operations remove or anonymize only the intended records without affecting other workspaces. • Scan results remain traceable across repeated scans through consistent fingerprints and model-version information. • Security-related events are recorded correctly in the append-only audit trail. • Session and invitation credentials are stored in a non-reusable form. • Historical results can be reconstructed correctly from the stored database records.

Special Considerations:	• Integration tests use isolated PostgreSQL 16 databases so that one test run does not affect another. • The test schema is created from the actual Alembic migration chain to keep it consistent with the production database structure. • Database access-control tests must run using the same restricted application role used in production. • External services may be stubbed during database-focused tests so that database behavior can be tested independently. • Neon-specific behavior is verified separately in the deployed environment.
-------------------------	--

3.1.2 Function Testing

Overview:

Function testing verifies that the main features and user workflows of CodeSage AI operate according to the Software Requirement Specification and produce the expected results for both valid and invalid user actions.

Technique Objective:	Verify that the functional requirements defined in the Software Requirement Specification are implemented correctly from the user's point of view. Testing covers the main user workflows, including authentication, repository connection, scan execution and cancellation, technical debt detection, scoring and prioritization, dashboard behavior, profile management, scan history, and role-based operations. Valid inputs should produce the expected results, while invalid or unauthorized actions should be rejected correctly.
Technique:	• Equivalence Partitioning and Boundary Value Analysis [3]: Used for repository URL validation, scoring-weight limits from 0.1 to 3.0, and the maximum of five scoring profiles. • State-Transition Testing: Used to verify valid scan lifecycle transitions, including idle, queued, running, done, error, and cancelled states. • Decision Table Testing: Used to verify authorization behavior against the role-operation permission matrix defined in <code>policy.json</code>. • Expected-Output Comparison: Used to compare generated results with manually calculated reference values, especially for scoring calculations and finding fingerprints.

Oracles:	• Successful sign-in, sign-out, and session behavior. • Correct workspace isolation and expected 404 responses for cross-workspace access. • Expected repository-validation error codes for invalid, private, or duplicate repositories. • Correct project, branch, dashboard, history, and profile behavior in the UI. • Correct scan lifecycle states and preservation of previous results after cancellation or failure. • Extracted CK metrics, process metrics, and comments match the expected fixture data. • Repeated scans produce consistent fingerprints and findings. • Rule, security, and SATD findings contain the expected source, category, severity, and evidence. • ML risk scores remain within the expected range and degraded-mode behavior occurs when the ML service is unavailable. • Scoring results match manually calculated priorities, health scores, and grades. • Dashboard, category, trend, Refactor-First, file-tree, and finding-detail views display the expected data. • Profile limits, weight boundaries, presets, and rescoring behavior match the defined requirements. • Role-based operations match the permissions defined in <code>policy.json</code>. • Critical security findings remain prioritized according to the defined security-priority rule.
Required Tools:	• pytest – Backend unit and integration testing. • FastAPI TestClient – Calls backend API endpoints during integration tests. • PostgreSQL 16 – Database used for backend integration testing • Docker and Testcontainers – Provide isolated PostgreSQL test environments. • Vitest – Web component and hook testing. • Testing Library – Tests frontend component behavior. • Playwright [8] – End-to-end browser testing. • Mock Service Worker – Provides contract-shaped API responses for browser tests. • axe-core – Accessibility checks during selected E2E tests. • GitHub Actions – Runs automated test suites in CI.

Success Criteria:	• Major functional requirements in the SRS behave as expected. • Valid user actions complete successfully and produce the expected results. • Invalid inputs and unauthorized actions are rejected with the correct error responses. • Scan lifecycle transitions occur correctly for successful, cancelled, failed, and repeated scans. • Technical debt findings, metrics, risk scores, and priorities match their expected values. • Dashboard, history, profile, and finding views display the correct information. • Repeated scans preserve consistent fingerprints and traceability. • Scoring calculations match manually verified reference values. • Role-based operations follow the permissions defined in <code>policy.json</code>. • No unresolved critical functional defects remain.
Special Considerations:	• Playwright tests use mocked API and authentication responses, so backend behavior is verified separately through integration testing. • External ML services are replaced with controlled stubs where required to keep tests repeatable. • End-to-end browser automation is primarily performed in Chromium. • Functional test results may depend on the current scoring configuration and approved model behavior. • Integration tests require PostgreSQL and additional services, so they take longer than unit tests.

3.1.3 User Interface Testing

Overview:

Verify that the web application shows the right data, handles every screen state and lets a user complete the main journeys. Automated tests are used wherever possible; a short manual check and a usability session cover what automation cannot judge.

Technique Objective:	Confirm that each main screen (Login, Projects, Dashboard and finding detail, Scan History, Profiles, Workspace and Team) renders the data returned by the API, shows a loading, a named empty and an error state, and can be operated by mouse and keyboard at a viewport of 1280 x 720 or larger.
----------------------	---

Technique:	• Component tests: Vitest [9] and React Testing Library [10] render each screen component against mock API responses served by MSW, covering the loading, empty, error and success states. • End-to-end tests: Playwright drives the main journeys in a Chromium browser: connect a repository, run and stop a scan, open the dashboard, switch branch, open a finding, load a past snapshot from Scan History and apply a scoring profile. They run against the MSW mock layer with a pre-seeded session, because a headless browser cannot complete the Asgardeo sign-in. • Accessibility: axe-core runs inside the Playwright suite on the main routes. Keyboard-only operation of the dashboard is covered by dedicated Playwright tests. • Manual check on the deployed site (https://codesageai.dev): real Asgardeo sign-in and sign-out, dark and light themes, 1280 x 720 with no horizontal scrolling, and the 401, 403, 404 and 409 messages. This is followed by a usability session in which 5 participants perform the primary journey without help.
Oracles:	• Each Playwright journey must reach its expected screen and text; a failed step fails the test and saves a screenshot. • Each component test asserts the exact state shown: skeleton, named empty message, error message with a next action, or data. • axe-core reports the WCAG 2.1 AA violations found on each checked page. • The manual check and the usability session are recorded on a checklist: pass or fail per item, and each participant's success and completion time.
Required Tools:	• Playwright - end-to-end browser automation (Chromium). • Vitest + React Testing Library [9][10] - component tests. • MSW (Mock Service Worker [11]) - mock API responses that follow the OpenAPI contract, shared by both test layers. • @axe-core/playwright - automated accessibility audit. • Chrome DevTools - manual checks of layout, cookies and network responses.

Success Criteria:	• All Playwright journeys and Vitest component tests pass. • Every list and panel has a loading, named empty and error state and no screen ends in a dead-end error. • Zero automated WCAG 2.1 AA [12] violations on the checked pages, and the primary journey can be completed by keyboard alone. • No horizontal page scrolling at 1280 x 720. • At least 4 of 5 usability participants complete the primary journey without help from the facilitator, with a median time of 5 minutes or less.
Special Considerations:	• Automated end-to-end tests run in Chromium only (the engine of Chrome and Edge). Firefox and Safari are checked manually on the primary journey. • The real Asgardeo sign-in cannot be automated, so it is covered by the manual check and by the deployment smoke test, which checks the sign-in hand-off on the live site. • Usability testing participants include general software developers, fellow students, and members of active agile teams.

3.1.4 Performance Profiling

Overview:

Measure how quickly the deployed system responds for a single user, and take the four measurements required by SAD Section 11, so that the load test results can be compared with a baseline.

Technique Objective:	Measure the single-user response times of the main API requests, and the time and resources used by one repository scan on the production k3s node, and compare them with SRS PERF-02, PERF-03, PERF-04 and PERF-11.
----------------------	--

Technique:	• API baseline: a k6 script with one virtual user requests the session, the project list, the branch list, the dashboard payload (GET /api/repos/{id}/health) and the scan history for 2 minutes, after one warm-up request. • Scan submission: the response time of starting a scan (POST /api/repos/{id}/scan) is read from the browser's network panel, and the progress display is observed until the scan ends. • Scan time per KLOC: three public Java repositories of different sizes, all within the configured limits are scanned once each. The duration comes from the start and completion times stored on each analysis attempt; the size in KLOC is counted with cloc at the scanned commit. • Resource use: CPU and memory of the worker, ML and API pods are sampled with kubectl top while a scan runs, to record the peak memory of one scan and therefore how many scan workers the 8 GB node can hold. • Database growth: the database size is read before and after one scan to give the database size per snapshot.
Oracles:	• k6 reports the median and 95th-percentile time of each request. • Scan start and completion times stored on the analysis-attempt record. • kubectl top values for the node and for each pod. • PostgreSQL database size before and after a scan.
Required Tools:	• k6 - scripted HTTP requests and response-time statistics. • kubectl top (metrics-server, included in k3s) - pod and node CPU and memory. • Neon SQL editor - scan durations and database size queries. • cloc - line count of each scanned repository. • Chrome DevTools - network timing of scan submission.
Success Criteria:	• Single-user requests that do not involve analysis complete within 1 second. • The dashboard payload returns within 2 seconds. • Starting a scan returns within 1 second, and the scan shows its stage and progress until it completes, fails or is stopped. • The four SAD Section 11 values are recorded together with their conditions: scan time per KLOC, maximum concurrent scans per worker, P95 dashboard-read latency (from the load test) and database size per snapshot.

Special Considerations:	• There is no separate performance environment. Measurements are taken on the production k3s node. • Requests are sent from a laptop over the internet, so the times include real network latency, as a user would experience it. • The Neon database pauses when idle, so a warm-up request is sent first to avoid measuring a cold start. • Each scan worker runs one scan at a time by design (Celery concurrency 1), so the maximum concurrent scans per worker is 1; the measurement records how much memory that one scan needs.
-------------------------	---

3.1.5 Load Testing

Overview:

Check that the system keeps its response times with the number of users the SRS specifies (50 users, PERF-06), and that scans requested beyond the workers' capacity wait in the queue without slowing the rest of the application.

Technique Objective:	Apply the SRS baseline workload of 50 concurrent users and a burst of scan requests to the deployed system, and observe response times, errors, queue behaviour and node resources.
Technique:	• Read load: the k6 script from Performance Profiling ramps to 50 virtual users over 2 minutes, holds 50 users for 6 minutes and ramps down over 2 minutes. Each virtual user loads the session, projects, branches, dashboard payload and scan history, then pauses 1 to 3 seconds, like a person reading. • Scan queue: scans are started on seven different repositories in one workspace. A workspace allows one running scan and at most five waiting scans, so the request beyond that limit must be refused with a clear message. • Concurrent scans: one scan is started in each of three different workspaces while KEDA scales the scan workers, to observe how many scans run at the same time on one node. • Operation under load: the read load keeps running during the scan tests, to confirm that the session, navigation, dashboards and history stay available while scans are queued.
Oracles:	• k6 prints pass or fail for each threshold: dashboard 95th percentile, other requests 95th percentile, and failed-request rate. • The scan-status responses show the running and queued scans, and the refused request returns 429 SCAN_QUEUE_FULL. • kubectl shows the number of worker pods, any Pending or restarted pods, and node CPU and memory. • Every accepted scan ends as done, failed or cancelled; none stays queued or running.

Required Tools:	• k6 - load generation with ramp-up stages and pass/fail thresholds. • kubectl (get pods -w, top) - KEDA scaling, pod restarts and resource use. • CodeSage-AI web application - starting the scans and following their progress.
Success Criteria:	• With 50 concurrent users, dashboard requests stay within 2 seconds at the 95th percentile and other requests within 1 second. • Fewer than 1% of requests fail, and no pod is restarted or killed for lack of memory during the test. • Extra scans are queued or clearly refused, never lost, and every accepted scan finishes. • The number of scans that ran at the same time is recorded. If fewer than three fit in the 8 GB node, this is reported as a deviation from PERF-07 together with the queued behaviour.
Special Considerations:	• Only one server is available, so it is limited to the SRS workload (50 users for about 10 minutes) and is not a breaking point test. • All virtual users share one test account and workspace, because the Asgardeo sign-in cannot be scripted for many users; sign-in itself is not load-tested. • Scans use a small, fixed set of public Java repositories, because connecting repositories is subject to GitHub API rate limits. • Dashboard timings are measured after the test project's scores have been calculated, and the profile is not changed during a run. Right after a profile change or a new scan, the API replies immediately with SCORE_PENDING while the scores are recalculated in the background, and the dashboard shows that it is calculating. The time until the new scores are ready is recorded separately.

3.1.6 Security and Access Control Testing

Overview:

Verify that only authenticated and authorized users can access the system and that no user can access another user's data.

Technique Objective:	Verify that protected resources can be accessed only by authenticated users and that each user can perform only the operations permitted by their role within their own workspace. Testing also checks session invalidation, protection of sensitive information, safe handling of invalid or unauthorized input, audit logging of security-related events, and isolation of data between workspaces.
----------------------	---

Technique:	• Role-based access testing: Each protected endpoint is tested using all defined roles and compared with the permissions in policy.json. • Cross-tenant testing: Users attempt to access resources belonging to another workspace. API requests must be blocked, and database RLS must prevent direct cross-workspace access. • Session security testing: Expired, invalid, signed-out, or incorrect session tokens are tested and must return 401 Unauthorized. • Secret leakage testing: Logs, audit records, responses, and database tables are checked to ensure that tokens, credentials, and other sensitive values are not exposed. • Privilege testing: Restricted database operations are attempted using the application role and must be rejected. • Input validation testing: Malformed URLs, invalid values, unknown fields, and invalid confirmation data must return appropriate 4xx responses. • Audit testing: Security-sensitive actions are performed and the resulting audit records are checked for completeness and immutability. • Security-header testing: Deployed responses are inspected for the required security headers. • OWASP-based security testing: Relevant OWASP Web Security Testing Guide scenarios will be used to test authentication, authorization, session management, input validation, security misconfiguration, injection, and information exposure.
------------	---

Oracles:	• Protected endpoints without a valid session return 401. • Role-based access decisions match policy.json. • Permission changes take effect on the next request. • Cross-workspace resources remain inaccessible through both the API and database RLS. • Unauthorized database operations are rejected. • Signed-out, revoked, or invalid sessions cannot be reused. • Session and invitation credentials are stored only in non-reusable form. • Sensitive tokens and secrets do not appear in logs, responses, database records, or audit records. • Invalid or malicious inputs return the expected 4xx responses and do not create unintended data. • Security events create the expected audit records. • Audit records remain append-only. • Error responses do not expose stack traces, SQL details, secrets, or internal addresses. • Authentication state validation rejects missing, tampered, or mismatched values. • Only explicitly public routes are accessible without authentication. • Required security headers are present on deployed responses.
----------	---

Required Tools:	• pytest – Unit and integration security testing. • FastAPI TestClient – Sends crafted requests with cookies, headers, and request bodies. • PostgreSQL 16 – Tests RLS policies and database privileges. • Docker and Testcontainers – Provide isolated PostgreSQL environments. • pytest caplog – Checks application logs for leaked sensitive information. • Vitest – Tests frontend route protection. • Browser developer tools – Manually inspect deployed security headers and behavior. • curl – Checks deployed response headers. • GitHub Actions – Runs automated security-related test suites in CI. • OWASP ZAP [16] – Performs dynamic security testing of the deployed application, including passive scanning, selected active vulnerability scans, and security-header inspection.
Success Criteria:	• All protected resources require valid authentication. • Each role can perform only the operations permitted by policy.json. • Workspace data remains isolated at both API and database levels. • Invalid, expired, signed-out, and revoked sessions are rejected correctly. • Sensitive credentials and tokens are not exposed or stored in reusable form. • Unauthorized database operations are rejected by the configured privileges and RLS policies. • Invalid and malicious inputs are rejected safely without unintended data changes. • Security-related actions are recorded correctly in the append-only audit trail. • Error responses do not expose internal implementation details or sensitive information. • Required security headers are present on the deployed application.

Special Considerations:	• Database security tests must use the restricted application role so that RLS and privilege enforcement match production behavior. • External services such as GitHub, Asgardeo, and the ML service are replaced with controlled stubs during automated testing. • Real credentials are not required for automated security tests. • Deployed security headers and TLS-related behavior must be checked in the deployed environment because they cannot be fully verified through in-process tests. • Security tests that inspect logs and audit records must use controlled test credentials and synthetic data only.
-------------------------	---

3.1.7 Failover and Recovery Testing

Overview: Verify that the system handles infrastructure failures gracefully and recovers automatically without data loss.

Technique Objective:	Verify that CodeSage AI can handle component failures without corrupting durable data and can be restored to an operational state within the 30-minute recovery target defined by REL-07. Testing also checks that failed or interrupted scans do not expose partial results, failures remain isolated, degraded services are handled correctly, and restored data remains valid and accessible.
Technique:	• Automated fault-injection testing: The real scan pipeline is executed against PostgreSQL 16 while selected external dependencies are replaced with controlled stubs. Failures are introduced during extraction, ML-service calls, timeout handling, cancellation, and database finalization. The resulting database state and scan status are then verified. • Manual recovery drills: Selected components in the deployed k3s environment are deliberately stopped or restarted to verify service recovery. These drills include API pod failure, worker restart, Redis restart, and ML-service failure. • Database recovery testing: Neon point-in-time recovery [17] and PostgreSQL dump-and-restore procedures are used to restore persistent data and validate the recovered database. • Post-recovery validation: After recovery, the application is checked for schema validity, sign-in, tenant isolation, dashboard access, and data consistency. • Recovery-time measurement: Recovery time is measured from the introduction of the failure until the application successfully serves a dashboard request again.

Oracles:	• Failed, cancelled, or timed-out scans leave no partial finalized snapshot. • Scan attempts end with the expected terminal status and failure information. • Previously completed results remain unchanged after a failure. • Failure of one repository or scan does not affect other repositories or workspaces. • Unavailable ML services result in degraded operation rather than incorrect successful output. • GitHub or Asgardeo failures return the expected upstream error responses. • Restarted Celery tasks do not create duplicate finalized snapshots. • Remaining API replicas continue serving requests when one API pod fails. • Failed worker, Redis, and ML components recover or fail cleanly without corrupting stored results. • Restored databases contain the expected rows, checksums, RLS policies, grants, and schema revision. • Tenant isolation and authentication continue to work after recovery. • Re-running the same commit after recovery produces the expected fingerprints and metrics. • Full-service recovery is completed within the 30-minute REL-07 target.
----------	--

Required Tools:	• pytest – Automated fault-injection and recovery tests. • PostgreSQL 16 – Persistent data store used during automated recovery testing. • Docker and Testcontainers – Provide isolated PostgreSQL test environments. • Stubbed Git, extractor, ML, and Redis dependencies – Introduce controlled and repeatable failures. • kubectrl – Stops, restarts, and monitors components in the k3s environment. • Neon console – Performs point-in-time database recovery. • pg_dump / pg_restore – Performs independent PostgreSQL backup and restore testing. • infra/scripts/backup-verify.sh – Supports backup and restore verification. • Alembic – Confirms the schema revision after database recovery. • Timestamps / stopwatch – Measures recovery time against REL-07.
Success Criteria:	• Failed, cancelled, timed-out, or interrupted scans do not leave partial finalized data. • Previously completed analysis results remain intact after failures. • Failures remain isolated and do not affect unrelated repositories, workspaces, or users. • External-service failures are handled using the expected error or degraded behavior. • Retried or redelivered tasks do not create duplicate finalized snapshots. • Failed application components restart or recover as expected. • Replicated API services remain available when a single instance fails. • PostgreSQL data can be restored successfully using the defined recovery procedures. • Restored data passes integrity, authorization, schema, and application-level checks. • Post-recovery scans produce consistent results. • The system is restored to an operational state within the 30-minute REL-07 target.

Special Considerations:	• Manual failover and recovery drills should be performed in a controlled production-like environment to avoid affecting active users or production data. • Database recovery tests should use a Neon recovery branch or isolated PostgreSQL instance rather than overwrite the active production database. • Redis contains transient queue and progress information, so some in-progress state may need to be recreated after failure. • Single-replica worker and ML components may cause temporary degraded operation while they restart. • Recovery-time compliance should only be claimed after a complete timed recovery drill has been performed and recorded. • The backup verification script should not be treated as final recovery evidence until its known issues are fixed.
----------------------------	---

3.1.8 Configuration Testing

Overview:

Verify the system works correctly across different browsers, operating systems, and environment configurations.

Technique Objective:	Verify that protected resources can be accessed only by authenticated users and that each user can perform only the operations permitted by their role within their own workspace. Testing also checks session invalidation, protection of sensitive information, safe handling of invalid or unauthorized input, audit logging of security-related events, and isolation of data between workspaces.
Technique:	• Application configuration testing: Important settings such as cookie options, identity-provider configuration, session behavior, and ML-service availability are varied to verify correct behavior under valid and invalid configurations. • Scoring configuration testing: The actual scoring YAML files are loaded and results are compared with manually calculated values. Boundary conditions for scoring weights and trust values are also checked. • Database configuration testing: The schema is created from a fresh installation and upgraded from older revisions containing existing data to confirm that both paths produce the expected final database state. • Deployment configuration testing: The application is tested in both Docker Compose and production-like k3s environments. Deployment smoke tests verify that the main application functions remain available. • Browser and display configuration testing: Main user journeys are tested automatically in Chromium at 1280 × 720 in both light and dark themes. Firefox, Edge, and Safari are checked manually. • Runtime compatibility testing: The backend test suite is run with supported Python versions to identify version-specific configuration issues. • Autoscaling configuration testing: KEDA worker scaling is observed under queued scan workloads to verify that worker replicas respond to queue demand.

Oracles:	• Missing or invalid application settings produce clear and safe errors. • Cookie behavior matches the configured domain and <code>Secure</code> settings. • Expired sessions are rejected with 401. • Unavailable or slow ML services result in the expected degraded behavior. • Scoring results match manually calculated values from the configured YAML files. • Weight and trust values remain within their configured limits. • Preset, custom, default, and project-specific scoring profiles are applied correctly. • Fresh database installation and migration upgrade paths produce the same expected schema, policies, and grants. • Existing data is preserved during supported database upgrades. • The local Docker Compose stack starts successfully and supports an end-to-end scan. • Production k3s components become ready and the deployment smoke test succeeds. • Main user journeys behave consistently across supported browsers. • Light and dark themes are applied correctly and remain selected after reload. • Required keyboard-accessibility checks pass and no serious axe violations are reported. • KEDA worker replicas scale according to queued workload and reduce when idle. • Backend tests pass under the supported Python versions.
----------	---

Required Tools:	• pytest – Runs backend tests with different application settings. • PostgreSQL 16 – Tests fresh database creation and migration upgrade paths. • Docker and Testcontainers – Provide isolated database environments. • Playwright – Runs automated Chromium E2E tests and deployment smoke tests. • axe-core [13] – Performs automated accessibility checks. • Chrome, Firefox, Edge, and Safari – Used for supported browser compatibility testing. • Docker Compose – Tests the local full-stack deployment configuration. • k3s on Linode – Provides the production-like deployment environment. • Neon PostgreSQL – Provides the production-like database environment. • KEDA and kubectl – Used to verify worker autoscaling and deployment behavior. • GitHub Actions – Runs automated configuration test suites in CI.
Success Criteria:	• Valid configurations allow the application to operate as expected. • Missing or invalid configurations fail safely with clear errors. • Cookie, session, identity-provider, and ML-service settings behave according to their configured values. • Scoring configurations produce the expected calculated results and enforce defined limits. • Fresh installation and database upgrade paths produce the expected schema without loss of existing data. • The application operates successfully in both local and production-like deployment environments. • Main user journeys work correctly in the supported browser configurations. • Light and dark themes behave correctly and persist as expected. • Required accessibility checks pass. • Worker autoscaling behaves according to the configured queue-based scaling rules. • Backend tests pass under the supported Python versions.

Special Considerations:	• Automated browser testing is primarily performed using Chromium; Firefox, Edge, and Safari require manual verification. • CI currently runs the backend on Python 3.12, while Python 3.13 is verified locally, so the two runtime versions do not currently have identical CI coverage. • Production-specific behavior such as ingress, autoscaling, and Neon integration must be verified in the production-like k3s environment. • Automated E2E tests may use mocked API behavior, so deployment smoke tests are required to verify integration with the real backend. • Scoring expectations depend on the currently configured calibration values and may need to be updated if the scoring configuration changes. • Cross-browser results obtained manually should be recorded separately from automated Chromium results.
-------------------------	--

3.1.9 Machine Learning Model Testing

Overview:

Verify that the ML Inference Service produces accurate, reliable predictions for both SATD detection and defect risk scoring.

Technique Objectives:	Verify that the ML models operate correctly throughout the prediction pipeline by validating the canonical 21-feature bug-risk input, class-to-file risk aggregation, SATD classification into supported categories, persistence of model versions and prediction provenance, safe handling of invalid artifacts or unavailable ML services, and project-separated evaluation to reduce cross-project leakage.
Technique:	• Offline model evaluation: ML-2 is evaluated using ested leave-one-project-out validation across five projects, using ROC-AUC, PR-AUC, and Brier score. SATD uses project-separated evaluation with classification and probability-based metrics. • Calibration evaluation: Project-aware sigmoid calibration is evaluated by comparing raw and calibrated probability metrics. • Dataset and leakage testing: Automated tests verify canonical columns, target derivation, missing/non-finite values, duplicate class identities, and exclusion of leakage-prone fields. • Feature-contract testing: Automated tests verify the exact ML-2 feature count, names, order, target classes, and compatibility of serialized model artifacts. • Prediction and aggregation testing: Class-level predictions, probability ranges, missing-history behavior, and noisy-OR file aggregation are verified using known inputs and outputs. • SATD contract testing: Automated tests verify supported categories, response mapping, comment extraction, and classification behavior. • Integration and persistence testing: Tests verify that class risk, file risk, SATD predictions, model versions, and provenance are persisted correctly. • Degraded-mode testing: Network failures, malformed responses, HTTP failures, and unavailable ML services are injected to verify controlled scan continuation.

Oracles:	• ML-2 input contains exactly 21 features in the canonical order. • ML-2 target classes are exactly {0,1}. • Class-level and file-level risk probabilities remain within [0.0, 1.0]. • Persisted SATD confidence remains within [0.0, 1.0]. • Missing or unexpected ML-2 features are rejected. • The loaded ML-2 artifact exposes predict_proba and the expected binary classes. • Noisy-OR aggregation produces the expected file probability • A file containing one class retains that class's probability as its file risk. • Missing Git history produces thirteen explicit zero-valued process features. • Inner and anonymous CK entities are excluded from ML-2 input. • SATD output is limited to code-design, requirement, documentation, test, or non-debt. • SATD does not produce the security category. • ML-service failures result in controlled degraded operation and do not prevent non-ML scan results from being finalized. • Persisted predictions reference the model version that produced them. • Database constraints enforce valid prediction ranges, category consistency, and prediction uniqueness.
----------	---

Required Tools:	• Python 3.12+ – ML training, evaluation, inference, and automated tests. • pytest – Automated model, feature-contract, client, pipeline, and persistence testing. • scikit-learn [19] – Random Forest, logistic calibration, TF-IDF, LinearSVC, and evaluation metrics. • pandas and NumPy – Dataset preparation, validation, and numerical evaluation. • joblib – Serialization and loading of trained model artifacts. • FastAPI and Pydantic – ML inference endpoints and request/response validation. • httpx – Communication between the API and ML service. • PyDriller [20] – Extraction of process/change metrics used by ML-2. • CK [21] – Extraction of Java class-level product metrics. • Tree-sitter [22] – Comment extraction for SATD classification. • PostgreSQL 16, Docker, and Testcontainers – Prediction persistence and database-constraint testing. • GitHub Actions – Automated ML-related tests in CI.
-----------------	---

Success Criteria:	• ML-2 receives exactly the canonical 21 features in the required order. • The loaded ML-2 artifact matches the current feature contract and provides binary probabilistic output for classes {0,1}. • All persisted risk and confidence values satisfy their defined database ranges. • SATD debt predictions contain a supported category, while non-debt predictions do not contain a debt category. • ML-1 never produces security as an SATD category. • Class predictions correspond to the classes supplied in the request. • Class-level predictions are aggregated into file-level risk using the defined noisy-OR calculation. • Model versions and prediction provenance are stored with successful predictions. • Process metrics do not use Git history occurring after the scanned commit. • Invalid or incompatible model artifacts and malformed ML responses are rejected safely. • ML-service failure does not prevent a scan from completing with available non-ML results. • Dataset preparation produces the expected canonical ML-2 dataset without missing, non-finite, or duplicate class observations. • Automated ML contract, pipeline, and persistence tests pass.
Special Considerations:	• ML-2 was trained and evaluated using five Java projects, so generalization to other projects or languages has not been established. • ML-2 predicts eligible Java classes and derives file-level risk using noisy-OR; it does not directly predict file risk. • Missing Git-history metrics are represented by zeros, so the model cannot distinguish unavailable history from genuine zero values. • SATD performance varies by project and debt category, with limited examples for some categories. • The SATD severity level is assigned by deterministic marker rules rather than predicted by the ML model. • The experimental calibrated SATD model is not the currently selected runtime model.

4. Deliverables

The testing will produce a small set of records that show what was tested and how well CodeSage AI met the targets in the SRS and SAD. The main deliverables are:

- Test logs from the automated tests
- User interface test details
- Performance and load test results
- ML model evaluation results
- Usability session notes
- Test evaluation summary report

4.1 Test Evaluation Summaries

We will test the parts of CodeSage AI that users depend on most:

- The web pages and the main user journey: signing in, connecting a repository, running a scan and reading the dashboard.
- The backend and the scan process, including stopping a scan and what happens when a scan fails.
- Data separation, so one team can never see another team's projects or results.
- Sign-in and permissions, so each role can only do what it is allowed to do.
- Speed, with one user and with 50 users at the same time.
- The accuracy of the two ML models.

After the final round of testing, the results will be brought together in one short summary report. It will say what was tested, how many tests passed and failed, which bugs were found and fixed, the key speed and accuracy numbers, and whether the system is ready for submission.

4.2 Reporting on Test Coverage

The automated tests run on every code change through GitHub Actions, so their logs are always up to date. The browser tests, the speed and load tests and the usability session are run once, in the final round of testing before submission.

Coverage will be reported in two simple ways: how much of the code the tests run through, and which SRS requirements have at least one test. Each test report will contain:

- Date
- Area tested
- Number of test cases run
- Number passed
- Number failed
- Pass percentage
- SRS requirements covered
- Comments

5. Risks, Dependencies, Assumptions, and Constraints

5.1 Risks

Risk	Likelihood	Impact	Mitigation	Contingency
Most automated browser tests use mocked API and authentication responses. Differences between the mocks and the deployed backend may therefore remain undetected.	Medium	High	Generate frontend types from the OpenAPI specification, test backend behaviour separately against PostgreSQL, and maintain a deployment smoke test that uses the real frontend and API.	Run the deployment smoke test and selected manual end-to-end journeys against the deployed environment before release.
GitHub, Asgardeo, or the ML service may return responses, errors, or timing behaviour that differ from the test stubs.	Medium	Medium	Model documented success, error, timeout, authentication, and rate-limit responses in automated tests. Test degraded-mode behaviour explicitly.	Perform controlled end-to-end checks using the real external services before release.
GitHub API rate limits may interrupt manual, integration, or load testing.	Medium	Medium	Use an authenticated test token where permitted, minimise repeated requests, and use a fixed set of small public repositories.	Pause or reschedule the affected tests and use stubbed responses for repeatable functional scenarios.
Automatic retry of transient scan failures required by REL-04 and application-level rate limiting required by SEC-12 are not fully implemented. Tests for these requirements cannot yet satisfy their acceptance criteria.	High	Medium	Record both requirements as known limitations and retain the planned tests so they can be activated when implementation is complete.	Apply rate limiting at the ingress or network layer where possible and report unmet requirements in the final test summary and release notes.
The scoring calibration constant k remains a placeholder, so absolute health scores and grades may change after calibration.	High	Medium	Test the scoring formula and boundary behaviour using the currently configured value, while keeping calibration-dependent expectations clearly identified.	Re-run scoring tests and update approved fixtures after the calibration exercise is completed.
Performance measured using reduced test data may not represent behaviour at the production volumes specified by DBR-32, DBR-33, and the performance requirements.	Medium	High	Run scheduled tests using the stated baseline dataset and concurrency level. Monitor response times, scan duration, resource consumption, and database query plans.	Optimise queries, add justified indexes, or reduce supported limits and document the measured capacity.
Failover or recovery drills could interrupt shared services or accidentally affect production data.	Low	High	Conduct drills in an isolated environment or Neon recovery branch, stop writes where required, and never overwrite the active production database.	Abort the drill, reconnect services to the original database, and investigate the failure before attempting another recovery.

Automated browser testing currently covers Chromium only, so browser-specific defects in Firefox, Edge, or Safari may remain undetected.	Medium	Low	Perform a manual compatibility check for critical user journeys on supported browsers.	Record browser-specific issues and prioritise them according to severity and user impact.
Database integration and end-to-end tests require additional services and take longer than unit tests, which may delay developer feedback and CI completion.	Medium	Low	Run unit tests and static checks first, execute targeted integration tests during development, and reserve the complete suite for CI or release validation.	Split long-running suites into separate CI jobs and parallelise independent test groups where practical.

5.2 Dependencies

The following dependencies must be available before testing begins.

Dependency	Required For
Docker and Testcontainers	Providing isolated PostgreSQL instances for database integration testing.
GitHub Actions	Running automated checks and test suites on pull requests and changes to the main branch.
GitHub, Asgardeo, and ML service	Real-service integration testing, deployment testing, and manual end-to-end testing.
Linode k3s cluster and Neon PostgreSQL	Smoke testing, performance testing, failover testing, and recovery testing.
Neon point-in-time recovery	Performing database recovery drills within the available recovery history.
SATD and bug-risk model artifacts	Running the deployed ML service and ML-related analysis.
pytest, Vitest, Playwright, axe-core, OWASP ZAP, and k6 [14]	Executing backend, frontend, accessibility, security, and performance tests as applicable.
Successful functional and data-integrity testing	Allowing performance, security, and recovery testing to proceed.

5.3 Assumptions

The following assumptions apply to the test plan:

1. The Docker-based PostgreSQL test environment is sufficiently compatible with the Neon PostgreSQL production environment for testing schema migrations, constraints, roles, grants, and Row-Level Security. Environment-specific behaviour will be checked separately during deployment testing.
2. Mock and stub responses represent the documented behaviour of GitHub, Asgardeo, the CodeSage API, and the ML service.
3. Version 1.0 analyses public GitHub repositories only; private-repository support is outside the test scope.
4. The primary supported client is a desktop browser with a viewport of at least 1280×720 . Smaller viewports are outside the formal acceptance scope unless stated otherwise.
5. `docs/api/openapi.yaml` is the agreed API specification used to generate and validate frontend API types.
6. Test data is synthetic or obtained from public repositories and does not contain private production data.
7. Valid test credentials, tokens, and deployed services will be available for scheduled integration, smoke, and recovery tests.
8. The production-like environment will be stable and available during the agreed testing windows.

5.4 Constraints

Constraint	Description
Budget and infrastructure	The project uses limited or low-cost infrastructure, restricting load-test duration, concurrency, retained environments, and recovery-drill frequency.
Security assessment	No independent third-party penetration test is planned. Security evaluation is limited to automated tests, authorization tests, dependency and static analysis, and tool-assisted dynamic scanning.
Observation period	Long-term reliability targets, including 99% monthly availability under REL-01 and the 500-hour mean time between failures under REL-08, cannot be demonstrated within the project test period. They require production monitoring over time.
External service limits	GitHub rate limits and other third-party usage policies restrict the number and frequency of tests using real services.
Browser coverage	Automated end-to-end coverage is limited to Chromium. Other supported browsers require manual verification.

6. References

- [1] CodeSage-AI Group 16, *Software Requirements Specification (SRS), Version 1.1*, Department of Computer Science and Engineering, University of Moratuwa, 2026.
- [2] CodeSage-AI Group 16, *Software Architecture/Design Document (SAD), Version 1.1*, Department of Computer Science and Engineering, University of Moratuwa, 2026.
- [3] International Software Testing Qualifications Board, *Certified Tester Foundation Level Syllabus, Version 4.0.1*, ISTQB, 2024. [Online]. Available: [ISTQB CTFL Syllabus v4.0.1](#). [Accessed: Sep. 27, 2026].
- [4] PostgreSQL Global Development Group, “PostgreSQL 16 Documentation,” *PostgreSQL*. [Online]. Available: [PostgreSQL 16 Documentation](#). [Accessed: Sep. 27, 2026].
- [5] AtomicJar, “Getting Started with Testcontainers for Python,” *Testcontainers*. [Online]. Available: [Testcontainers for Python](#). [Accessed: Sep. 27, 2026].
- [6] SQLAlchemy Authors, “Alembic Documentation.” [Online]. Available: [Alembic Documentation](#). [Accessed: Sep. 27, 2026]. [Alembic](#)
- [7] FastAPI, “Testing,” *FastAPI Documentation*. [Online]. Available: [FastAPI Testing Documentation](#). [Accessed: Sep. 27, 2026]. [FastAPI](#)
- [8] Microsoft, “Playwright Documentation.” [Online]. Available: [Playwright Documentation](#). [Accessed: Sep. 27, 2026].
- [9] Vitest, “Getting Started,” *Vitest Documentation*. [Online]. Available: [Vitest Documentation](#). [Accessed: Sep. 27, 2026]. [Vitest](#)
- [10] Testing Library, “React Testing Library.” [Online]. Available: [React Testing Library Documentation](#). [Accessed: Sep. 27, 2026]. [Testing Library](#)
- [11] Mock Service Worker, “MSW Documentation.” [Online]. Available: [Mock Service Worker Documentation](#). [Accessed: Sep. 27, 2026].
- [12] World Wide Web Consortium, “Web Content Accessibility Guidelines (WCAG) 2.1,” *W3C Recommendation*. [Online]. Available: [WCAG 2.1](#). [Accessed: Sep. 27, 2026]. [W3C](#)
- [13] Deque Systems, “axe-core: Accessibility Testing Engine for Websites and Other HTML-Based User Interfaces.” [Online]. Available: [axe-core Documentation](#). [Accessed: Sep. 27, 2026]. [GitHub](#)
- [14] Grafana Labs, “Grafana k6 Documentation.” [Online]. Available: [Grafana k6 Documentation](#). [Accessed: Sep. 27, 2026]. [Grafana Labs](#)
- [15] OWASP Foundation, *OWASP Web Security Testing Guide, Version 4.2*. [Online]. Available: [OWASP WSTG v4.2](#). [Accessed: Sep. 27, 2026]. OWASP currently identifies v4.2 as the released version of the WSTG. [OWASP Foundation](#)
- [16] ZAP, “ZAP Documentation.” [Online]. Available: [ZAP Documentation](#). [Accessed: Sep. 27, 2026].

ZAP

[17] Neon, “Point-in-Time Restore,” *Neon Documentation*. [Online]. Available: [Neon Documentation](#). [Accessed: Sep. 27, 2026].

[18] Kubernetes Event-driven Autoscaling, “KEDA Documentation.” [Online]. Available: [KEDA Documentation](#). [Accessed: Sep. 27, 2026]

[19] scikit-learn Developers, “scikit-learn User Guide.” [Online]. Available: [scikit-learn User Guide](#). [Accessed: Sep. 27, 2026].

[20] D. Spadini, M. Aniche, and A. Bacchelli, “PyDriller: Python framework for mining software repositories,” in *Proc. 26th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*, 2018, pp. 908–911.

[21] M. Aniche, “CK: Code Metrics for Java Code by Means of Static Analysis,” 2015. [Online]. Available: [CK Java Metrics Tool](#). [Accessed: Sep. 27, 2026]. The project specifically provides class- and method-level Java metrics through static analysis. [GitHub](#)

[22] Tree-sitter Contributors, “Tree-sitter Documentation.” [Online]. Available: [Tree-sitter Documentation](#). [Accessed: Sep. 27, 2026].